

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

Projekt

**PRIMJENA VACUUM FILTERA PRI
TRAŽENJU SLUČAJNIH PODNIZOVA U E.
COLI GENOMU**

Matheo Kesar

Nika Valić

Voditeljica:

prof. dr. sc. Mirjana Domazet-Lošo

Zagreb, svibanj, 2026

Sadržaj

1. Uvod	1
2. Implementacija	2
2.1. Opis rada Vacuum Filtera	2
2.2. Funkcije umetanja, brisanja i pretraživanja	4
2.2.1. Funkcija umetanja	4
2.2.2. Funkcija brisanja	5
2.2.3. Funkcija pretraživanja	6
2.3. Parsiranje E. coli genoma	7
2.4. Rezultati	9
3. Zaključak	14
4. Literatura	15

1. Uvod

Upiti o približnoj pripadnosti su prostorno učinkovite podatkovne strukture koje odlučuju ako se upitni podatak nalazi u velikom skupu elemenata. Često se ovakve strukture koriste kod sustava poput Googlea, Bitcoina i sl. Najčešći primjeri ovakvih struktura su Bloom filteri. Kao najkorištenija podatkovna struktura oni također omogućavaju mali udio lažno pozitivnih rezultata uz dobru prostornu učinkovitost, no ne dopuštaju brisanje stavki skupa. Kako bi se unaprijedio Bloom filter, predstavljen je Cuckoo filter koji će omogućavati brisanje, no zakazuju kada veličine tablica u prostoru nisu potencija broja 2 te kad se ubacuju duplikati u skup.

Za razliku od postojećih podatkovnih struktura, prednosti vacuum filtera je učinkovitost korištene memorije te provedba vrlo malog broja lažno pozitivnih rezultata. Koriste do 25% manje prostora od ostalih struktura te pružaju veću propusnost kod umetanja i pretraživanja. Problem za kojeg se može koristiti vacuum filter je pretraživanje slučajnih podnizova *E. coli* genoma. Korištenje vacuum filtera poželjno je jer će za veliku podatkovnu strukturu poput genoma bakterije omogućuje da se više podataka nalazi u brzoj memoriji, što je ključno performanse. Također se koristi IUPR (eng. *Instant Updates and Periodical Reconstructions*) što omogućuje da učinkovito obrađuje umetnute duplikate i brisanje tako da rekonstruira filtere u pozadini. U ovom radu implementirane su funkcije umetanja, brisanja i pregledavanja.

2. Implementacija

U ovom poglavlju će se detaljnije opisati kako funkcionira primjena vacuum filtera na E. coli genom za pretraživanje slučajnih podnizova veličine k .

2.1. Opis rada Vacuum Filtera

Vacuum filter je tablična AMQ podatkovna struktura koja pohranjuje otiske prstiju (*eng. fingerprints*) elemenata kako bi odredili pripadnost s visokom učinkovitošću i brzom memorijom. Funkcionira tako da organizira otiske prstiju u tablicu s *buckets* od kojih svaki sadrži 4 mjesta.

Kada želimo ubaciti ili tražiti neki element, prvo će hash funkcija stvoriti reprezentacija stavke, tj. otisak prsta. Taj podatak će onda hash funkcija mapirati element na prvi kandidat za bucket. Koristi mapping funkciju (1)

$$\max(x, m) = (x \cdot m) \gg 32 \quad (1)$$

m - veličina tablice

Kako bi se održala razina lokalnost zato dijelimo tablicu na više jednakih dijelova, buckets. Dva kandidatski bucketa trebaju biti u istom dijelu memorije zbog lokalnosti. Za svaku stavku x se računaju indeksi 2 alternativna bucketa (2):

$$B_2(x) = \text{Alt}(B_1(x), f) = B_1(x) \oplus (H'(x) \bmod L) \quad (2)$$

L - broj uzastopnih bucketa

Ova formula omogućava filteru da pronađe drugi bucket po otisku prsta i prvog kandidata.

Uvodi se koncept alternativnih raspona kako bi se uravnotežio kompromis faktora opterećenja i lokalnosti podataka. Faktor opterećenja je omjer popunjenih mjesta u tablici i svih mjesta.

Za razliku od ostalih filtera, vacuum filter koristi višerasponsku alternativnu funkciju (*eng. Multi-range alternate function*) za izračun drugog kandidata za bucket.

Vacuum filter koristi 4 alternativna raspona. Manji raspon osigurava najbolju lokalnosti zbog blizine bucketa. Veći će raspon osigurati bolji raspon opterećenja jer će elementi biti

pravilno raspodijeljeni. Veličine rasporeda se računaju prvo algoritmom LoadFactorTest koji provjerava je li AR moguć te RangeSelection koji nalazi minimalni AR, kako bi faktor opterećenja bio oko 95%.

```
bool LoadFactorTest(int items, float a, float r, int L, int slots){
    // aproximation of the load... idk
    // items      - maximum num of items
    // a           - target load factor (around 96%)
    // r           - multiplier for a - for a more certain prediction (around 95%)
    // L           - alternate range - number of sequential buckets
    // slots       - number of slots per bucket

    // num of buckets (multiple of L)
    int m = ceil(items / a / slots / L) * L;
    // target number of entries
    int N = slots * m * r * a;
    // number of chunks
    int c = m / L;
    // min chunk capacity
    float P = 0.97 * slots * L;
    // max load estimator - "Balls into Bins" problem...
    float D = (double)N/c + 3.0/2*sqrt(2.0*N/c*log(c));

    return (D < P) ? true : false;
}

int RangeSelection(int items, float a, float r, int slots){
    int L = 1;
    while (LoadFactorTest(items, a, r, L, slots) == false) L *= 2;
    return L;
}
```

Slika 1: Kod za algoritme LoadFactorTest i RangeSelection

U ovoj implementaciji nije implementirano polu-sortiranje te se otisci prstiju pohranjeni direktno u tablicu bez sortiranja.

2.2. Funkcije umetanja, brisanja i pretraživanja

2.2.1. Funkcija umetanja

Funkcija umetanja prvo računa otisak prsta elementa pomoću hash funkcije te određuje 2 kandidatska bucketa, B_1 i B_2 . Vacuum filter za pretraživanje koristi kombinaciju DFS i BFS algoritma. Ako ima neki bucket slobodno mjesto, otisak prsta se automatski tamo upisuje. Ako su oba kandidatska bucket-a puna, onda se provodi proces brisanja. Filter će prvo pogledati sve buckete i ako ima slobodno mjesto, otisak prsta se premješta tamo i novi element. Ako nema slobodnih mjesta, nasumično se odabere otisak prsta, zamjenjuje se s novim elementom i traži se novo mjesto za izbačeni otisak prsta u alternativni bucket. Ovaj postupak će se ponavljati dok ne pronađe slobodno mjesto ili dok se ne dosegne $maxEvict$. Ako dođe do maksimalnog broja izbacivanja i umetanje nije uspjelo, sve se poništi.

Algorithm 5: Vacuum Filter : Insert(x)

```
 $f = H'(x)$  //  $H'$  is the fingerprint function;  
 $B_1 = H(x)$ ,  $B_2 = \text{Alt}(B_1, f)$ ;  
if  $B_1$  or  $B_2$  has an empty slot then  
    put  $f$  into the empty slot ;  
    return Success;  
end  
Randomly select a bucket  $B$  from  $B_1$  and  $B_2$  ;  
for  $i = 0$ ;  $i < \text{MaxEvicts}$ ;  $i++$  do  
    // Extend Search Scope  
    foreach fingerprint  $f'$  in  $B$  do  
        if Bucket  $\text{Alt}(B, f')$  has an empty slot then  
            put  $f$  to the original slot of  $f'$  ;  
            put  $f'$  to the empty slot ;  
            return Success ;  
        end  
    end  
    Randomly select a slot  $s$  from bucket  $B$  ;  
    Swap  $f$  and the fingerprint stored in the slot  $s$  ;  
     $B = \text{Alt}(B, f)$  ;  
end  
return Fail ;
```

Slika 2: Algoritam umetanja[1]

2.2.2. Funkcija brisanja

Funkcija brisanja, isto kao i funkcija umetanja, za element prvo računa otisak prsta i kandidatske buckete, B1 i B2. Krenu se pretraživati sva mjesta u B1 i B2 dok se ne pronade jednaka vrijednost otiska prsta kao i za željeni element. Ako se nađe otisak, na to mjesto se upisuje 0 (prazno mjesto).

```
template <typename T>
bool VacuumFilter<T>::del(uint64_t x)
{
    /*
     * f = fingerprint(x)
     * b1 = hash(x)
     * b2 = alt(b1, f)
     *
     * if f in b1 or b2:
     *     remove f from one of them
     *     return true
     * return false
     */

    T f = fingerprint(x);
    unsigned int b1 = pos_hash(x);
    unsigned int b2 = alt(b1, f);

    for (int i = 0; i < this->m; i++){
        if (table[b1*this->m + i] == f){
            table[b1*this->m + i] = 0;
            this->filled_cells--;
            return true;
        }
        if (table[b2*this->m + i] == f){
            table[b2*this->m + i] = 0;
            this->filled_cells--;
            return true;
        }
    }
    return false;
}
```

Slika 3: Kod za funkciju brisanja

2.2.3. Funkcija pretraživanja

Kod pretraživanja elementa trebaju se izračunati kandidatski bucketi B1 i B2. Ako bi otisak prsta traženog elementa bio nađen u jednom od 2 bucketa, pretraživanje je uspješno.

```
template <typename T>
bool VacuumFilter<T>::lookup(uint64_t x)
{
    /*
        f = fingerprint(x)
        b1 = hash(x)
        b2 = alt(b1, f)

        if f in b1 or b2:
            return true
        return false
    */

    T f = fingerprint(x);
    unsigned int b1 = pos_hash(x);
    unsigned int b2 = alt(b1, f);

    for (int i = 0; i < this->m; i++){
        if (table[b1*this->m + i] == f){
            return true;
        }
        if (table[b2*this->m + i] == f){
            return true;
        }
    }
    return false;
}
```

Slika 4: Kod za funkciju pretraživanja

2.3. Parsiranje E. coli genoma

E. coli genomi se učitavaju iz FNA datoteke koja se sastoji od zaglavlja i slijedom nukleotida. Kako bi izvukli podnizove iz podataka prolazimo kliznim prozorom veličine k kroz cijelu sekvencu. Dobivene podnizove provodimo kroz hash funkciju MurMurHash64. Vrijednosti koje će hash funkcija vratiti u k-mere genoma, podnizovi nukleotida duljine k, koje se ubacuju u vacuum filter.

Kako bi se testirali lažno pozitivni rezultati, generiraju se nasumični brojevi koji nisu u skupu k-mera genoma. Brojevi koji se generiraju očekuju dati negativne rezultate, a svaki pozitivan se broji kao lažno pozitivan. Za generiranje tih brojeva koristi se Mersenne Twister, mt19937_64.

Uz testiranje implementacije genoma E. coli, generirali su se “umjetno” generirani podatci.

```
vector<uint64_t> extract_kmer(const string& file_path, int k){
    ifstream file(file_path);
    string seq = "";
    string line;

    if(!file.is_open()){
        cout << "Error opening file.";
        return {};
    }

    while(getline(file, line)){
        if(line.empty()){
            continue;
        }
        if(line[0] == '>'){
            continue; // Skipping the header
        }
        seq += line;
    }
    file.close();

    vector<uint64_t> kmers;
    for(int i = 0; i <= (int)seq.size() - k; i++){
        uint64_t kmer_hash = hash_kmer(seq, i, k);
        if(kmer_hash != 0){
            kmers.push_back(kmer_hash);
        }
    }

    return kmers;
}
```

Slika 5: kod za ekstrakciju k-mera

```

vector<uint64_t> generate_random_kmer(const vector<uint64_t>& existing_kmer, int num){
    unordered_set<uint64_t> existing_set(existing_kmer.begin(), existing_kmer.end()); // All values in our existing k-mers
    cout << "Repeated kmer hashes: " << existing_kmer.size() - existing_set.size() << endl;
    vector<uint64_t> random_kmers;
    int seed = random_device{}();
    mt19937_64 rnd(seed); // Pseudo-random generator of 64-bit numbers

    while((int)random_kmers.size() < num){
        uint64_t val = rnd();
        if(val == 0){
            continue; // 0s are reserved for empty slots
        }

        if(existing_set.find(val) == existing_set.end()){
            random_kmers.push_back(val); // We're only adding values that aren't the the existing k-mers
        }
    }
    return random_kmers;
}

```

Slika 6: Generiranje nasumičnih k-mera

```

vector<uint64_t> artificial_data(int n){
    vector<uint64_t> data;
    int seed = random_device{}();
    mt19937_64 rnd(seed);

    while((int)data.size() < n){
        uint64_t val = rnd();
        if (val != 0){
            data.push_back(val);
        }
    }
    return data;
}

```

Slika 7: Generiranje “umjetno” generiranih podataka za testiranje VF-a

2.4. Rezultati

Rezultati implementacije su spremljeni u results.csv gdje su zabilježeni podaci poput tip podataka, k, udio lažno pozitivnih rezultata, propusnost umetanja i pretraživanja u *mops*, faktor opterećenja i bitovi elementa.

data_type	k	false_positive_rate	insert_throughput [MOPS]	lookup_throughput [MOPS]	load_factor	bits_per_item
ecoli	10	2.7e-05	0.237869	61.7565	0.621355	25.7502
ecoli	20	0.0001	19.5457	62.7834	0.943671	16.9551
ecoli	50	0.000106	28.3579	69.0854	0.944203	16.9455
ecoli	100	0.000116	29.8585	73.6447	0.944231	16.9451
ecoli	200	0.000111	30.2997	73.3334	0.944231	16.9451
artificial	0	0.000125	39.2584	75.2867	0.897576	17.826

Faktor opterećenja je kroz testiranje vacuum filtera dokazan da postiže 97% s vjerojatnost od 99%, stoga se faktor opterećenja fiksira na 95% i odabire se AR vrijednost takva da broj stavki nikad neće prijeći 97% kapacitet bloka. Naši rezultati pokazuju da se faktor opterećenja kreće oko 95% za $k \geq 20$.

Udio lažnih podataka je iznimno male vrijednosti jer se koristi vrijednost za gornju vrijednost: [1]

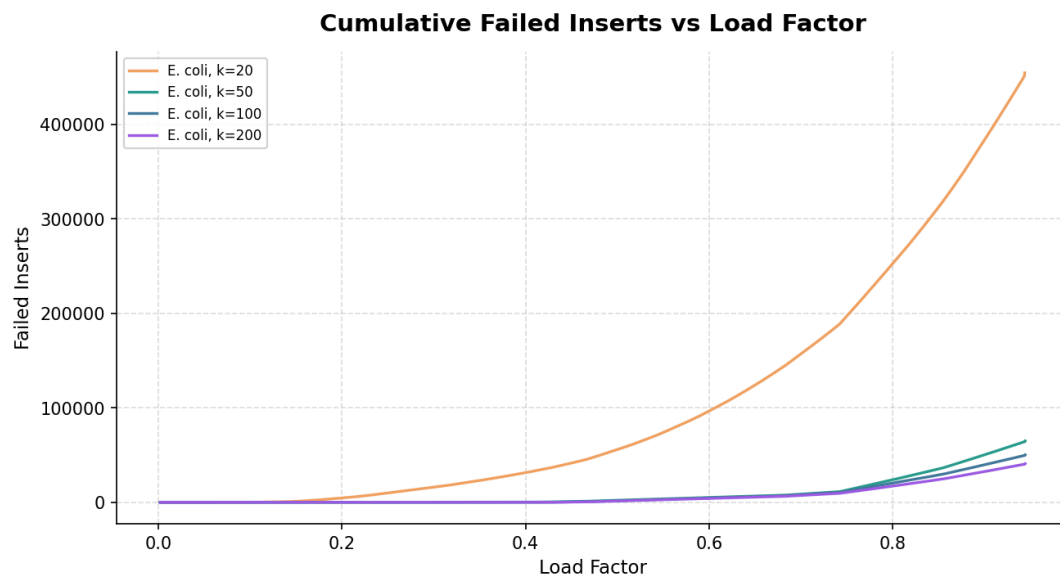
$$\epsilon = 1 - (1 - 1/2^l)^{2b\alpha}$$

l - duljina otiska prsta, koristimo 16-bitne otiske

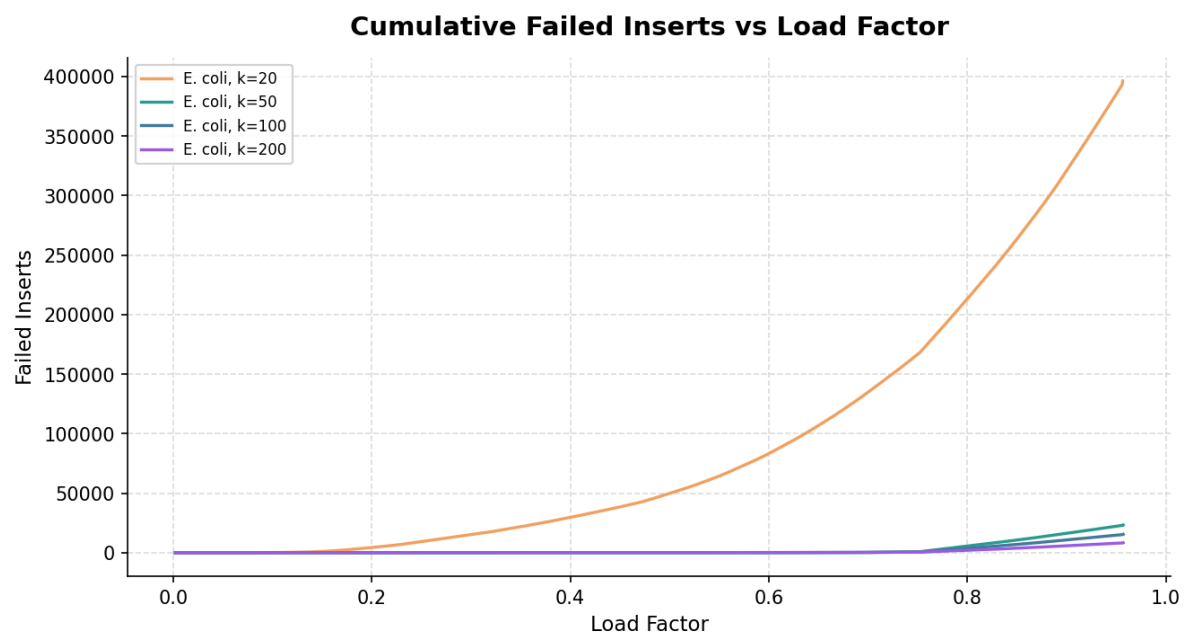
b - broj mjesta u bucketu

α - faktor opterećenja

Vacuum filter je davao najgore rezultate za $k = 10$ zbog puno sličnih/duplikata k-mera te jednakih hasheva koji se pokušavaju smjestiti na ista mjesta bucket-a.

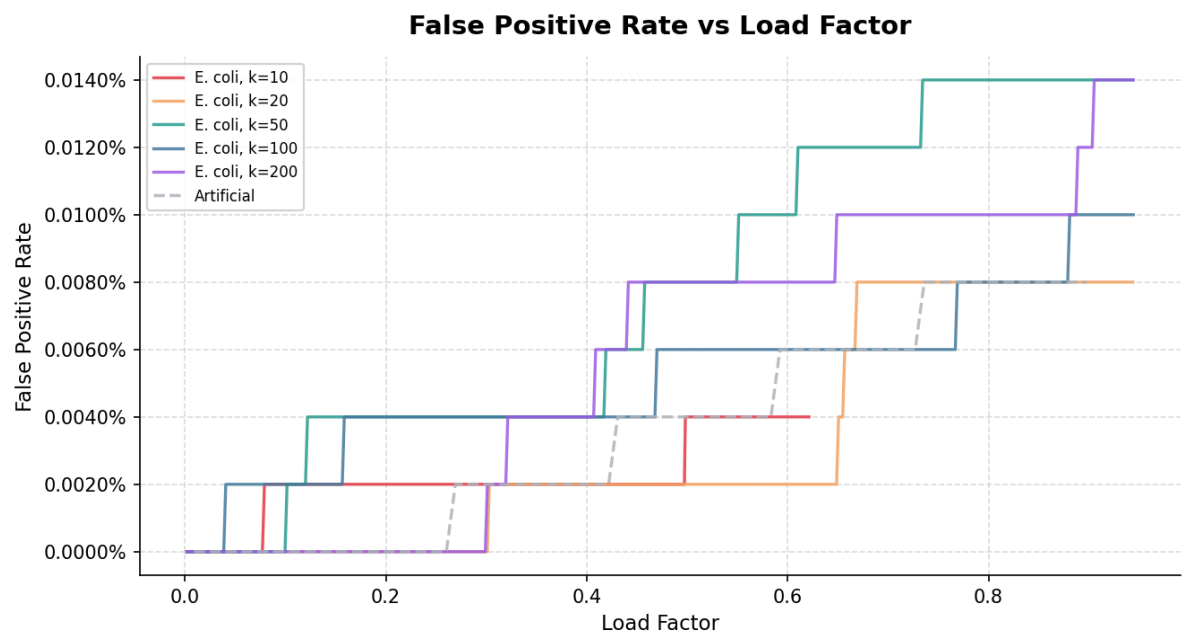


Slika 8: Neuspješna umjetanja naspram faktor opterećenja, naša implementacija

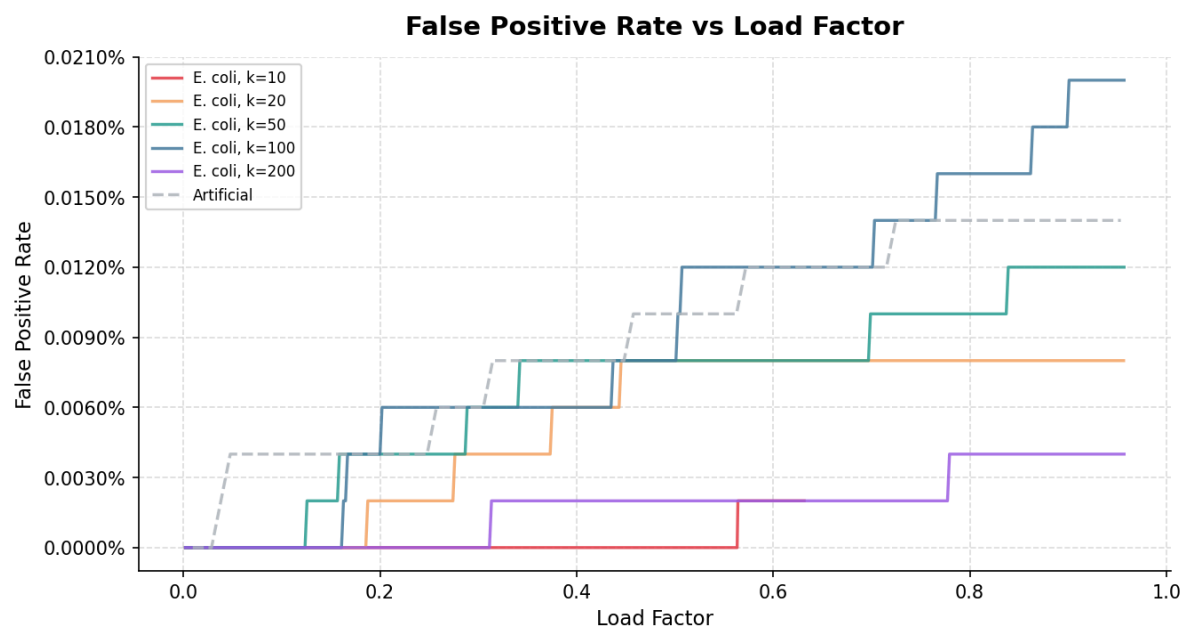


Slika 9: Neuspješna umetanja naspram faktor opterećenja, originalna implementacija

Za $k = 20$ se obje implementacije ponašaju slično, neuspješno umetanje kreće oko 0.2 faktora opterećenja i povećava se s faktorom. Za k od 50 do 200 naša implementacija ima više neuspjeha.

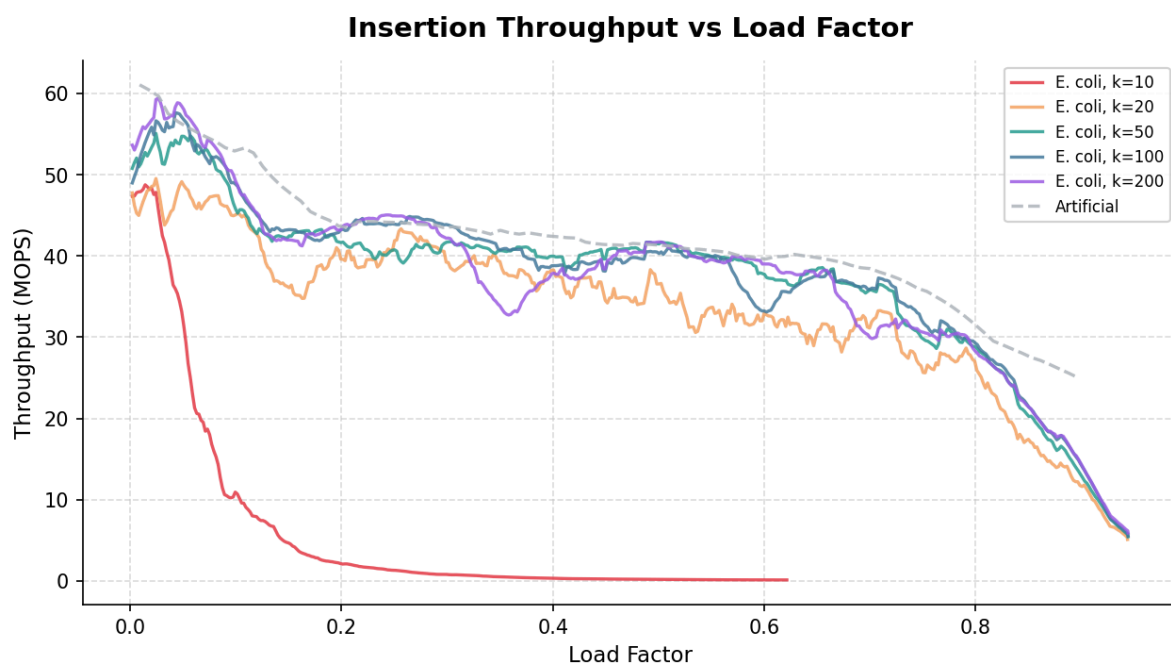


Slika 10: Udio lažno pozitivnih rezultata naspram faktora opterećenja, naša implementacija

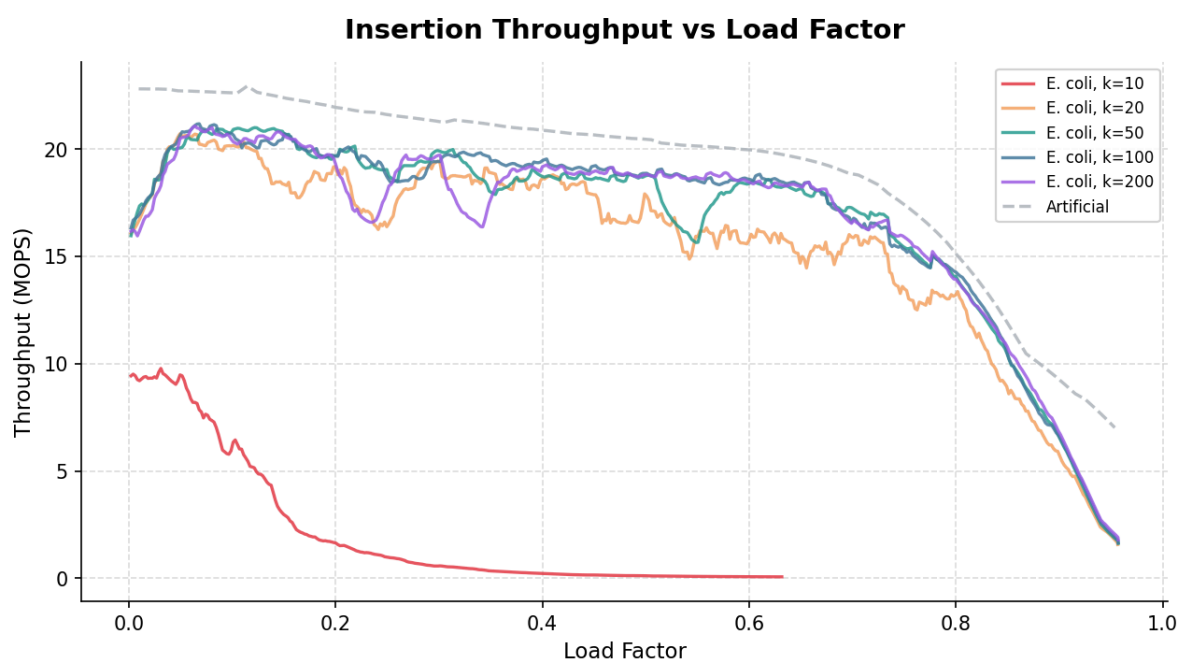


Slika 11: Udio lažno pozitivnih rezultata naspram faktora opterećenja, originalna implementacija

Stepenasti oblik jer se FPR mjeri za svakih 10000 umetanja. Umjetno generirani podaci imaju viši FPR jer su ravnomjernije raspoređeni u tablici te se stvara više kolizija.

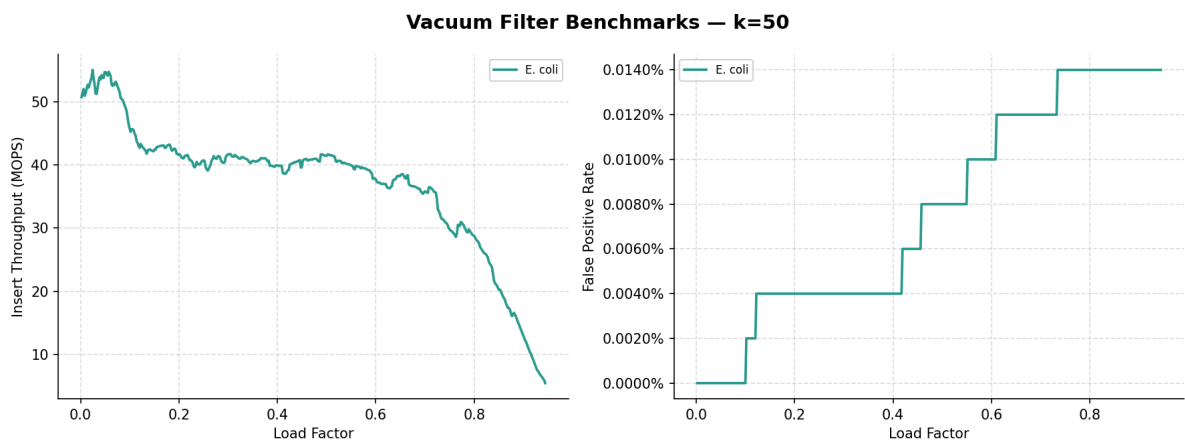


Slika 12: Propusnost umetanja naspram faktora opterećenja, naša implementacija

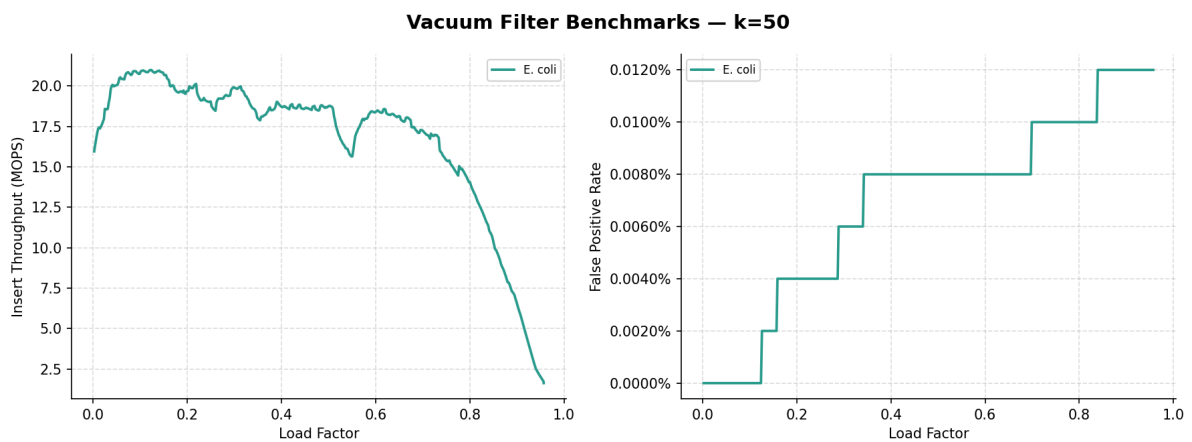


Slika 13: Propusnost umetanja naspram faktora opterećenja, originalna implementacija

Naša implementacija postiže višu početnu propusnost, ali neravnomjerno pada. Originalna implementacija ima manju početnu vrijednost propusnosti, ali stabilnije pada. Za $k = 10$ jako brzo pada zbog puno neuspješnih umetanja duplikata.



Slika 14: Faktor opterećenja i propusnost umetanja / stopa lažno pozitivnih rezultata, naša implementacija



Slika 15: Faktor opterećenja i propusnost umetanja / stopa lažno pozitivnih rezultata, originalna implementacija

Naša implementacija pokazuje nestabilnu propusnost i FPR raste pri nižem faktoru. Dok za originalnu implementaciju propusnost je stabilnija i FPR raste postupnije.

3. Zaključak

Vacuum filter je vrsta AMQ podatkovnih struktura koji je brži i učinkoviti od ostalih prijašnjih AMQ struktura. Uspješno je implementiran za traženje slučajnih podnizova u genomu bakterije E. coli. Implementirane su metode umetanja, koja kombinira BFS i DFS za postizanje većeg faktora opterećenja i bolju propusnost, uklanjanja te pretraživanja. Testiranja su provedena na genomu E. coli s različitim vrijednostima k (10, 20, 50, 100 i 200) te na umjetno generiranim podacima. Implementacija VF-a pokazuje da je faktor opterećenja oko 95%, što odgovara originalnoj implementaciji, udio lažno pozitivnih rezultata je vrlo malen za 16-bitne otiske prstiju te su propusnosti u skladu s originalnom implementacijom. Glavna razlika se pojavila kod korištene memorije, gdje naš VF koristi više memorije.

4. Literatura

- [1] Wang, Minmei, et al. "Vacuum Filters: More Space-Efficient and Faster Replacement for Bloom and Cuckoo Filters." *Proceedings of the VLDB Endowment*, vol. 13, no. 2, Oct. 2019, pp. 197–210. DOI.org (Crossref), <https://doi.org/10.14778/3364324.3364333>.
- [2] Fan, Bin, et al. "Cuckoo Filter: Practically Better Than Bloom." *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies* [Sydney Australia], 2014, pp. 75–88. DOI.org (Crossref), <https://doi.org/10.1145/2674005.267499>